

Payment Service — D1 e percorso verso un sistema di pagamento reale

Documento di riferimento del lavoro svolto nel progetto Spring Boot microservices. Descrive la trasformazione dello stato del pagamento in enum, la simulazione deterministica SUCCESS/FAILED e come questa struttura potrà essere evoluta in futuro verso un provider reale.

1. Situazione iniziale

Il Payment Service utilizzava uno stato rappresentato da una String. La creazione di un pagamento impostava lo stato direttamente a PENDING. Questo approccio era semplice ma non impediva valori errati come CREATED scritto male, né permetteva al compilatore di controllare l'insieme degli stati validi.

2. D1 — PaymentStatus enum

È stato introdotto l'enum PaymentStatus nel package entity con i valori previsti dal flusso:

PENDING, SUCCESS, FAILED

Nell'entity Payment il campo stato è ora tipizzato come PaymentStatus e persistito con @Enumerated(EnumType.STRING). Anche PaymentDTOOutput usa PaymentStatus.

3. Perché EnumType.STRING

Con EnumType.STRING il database memorizza il nome dell'enum, per esempio PENDING o SUCCESS, anziché l'indice numerico dell'enum. È una scelta più robusta perché l'aggiunta o il riordino dei valori non cambia il significato dei dati già presenti.

4. Simulazione deterministica dell'esito

Per rispettare l'esigenza della traccia di poter testare in modo prevedibile entrambi i rami, è stata separata la logica di simulazione dal PaymentService tramite l'interfaccia PaymentProcessor e l'implementazione SimulatedPaymentProcessor.

Struttura attuale

PaymentService → PaymentProcessor → SimulatedPaymentProcessor

Regola della simulazione

Importo	Esito
≤ 10.000,00	SUCCESS
> 10.000,00	FAILED

Il PaymentService non contiene direttamente la regola della soglia: chiede al PaymentProcessor di elaborare il pagamento e salva il PaymentStatus restituito. Questo rende la simulazione sostituibile.

5. Decisione attuale su FAILED

Un pagamento simulato che produce FAILED viene comunque registrato come risorsa. Il POST /api/payments può quindi rispondere 201 Created con stato FAILED. La gestione di un errore applicativo come PaymentFailedException è stata lasciata a una fase successiva, quando Order Service dovrà chiedere al Payment Service di eseguire un pagamento e reagire al suo esito.

6. Test manuali effettuati con Bruno

Scenario	Input	Atteso
SUCCESS normale	importo = 100,00	201 + SUCCESS

SUCCESS al limite	importo = 10.000,00	201 + SUCCESS
FAILED	importo = 10.000,01	201 + FAILED

7. Come evolvere verso un sistema reale

La struttura attuale è stata pensata per permettere di sostituire il simulatore con un provider reale senza spostare la logica di pagamento dentro il controller o dentro Order Service.

Architettura futura

Order Service → Payment Service → PaymentProcessor → Provider reale (es. Stripe/altro provider)

Il provider reale implementerà lo stesso contratto PaymentProcessor. In questo modo il PaymentService continuerà a coordinare il caso d'uso, mentre l'adapter/provider si occuperà della comunicazione esterna.

8. Cosa cambierà concretamente

Adesso	Futuro
SimulatedPaymentProcessor	ProviderPaymentProcessor / adapter
Regola importo ≤ 10.000	Risposta reale del provider
Esito calcolato localmente	Esito restituito dall'API del provider
Pagamento sincrono simulato	Possibile flusso asincrono con webhook
PENDING/SUCCESS/FAILED locali	Stati allineati al dominio e mappati dal provider

9. Aspetti necessari per un vero sistema di pagamento

- **Provider reale:** integrazione tramite API/SDK e credenziali gestite in modo sicuro.
- **Idempotenza:** evitare che un retry crei due addebiti per lo stesso ordine.
- **Webhooks:** ricevere aggiornamenti asincroni dal provider e verificare la firma delle notifiche.
- **Separazione dei dati:** non memorizzare dati sensibili delle carte nel nostro database se non strettamente necessario.
- **Gestione errori:** distinguere errori tecnici, pagamento rifiutato, timeout e stati temporanei.
- **Audit:** mantenere gli identificativi della transazione del provider e una cronologia utile per diagnosi e riconciliazione.
- **Sicurezza:** secrets fuori dal codice, HTTPS, autorizzazione tra servizi e logging senza dati sensibili.
- **Stati:** progettare le transizioni consentite, evitando cambiamenti arbitrari dello stato.

10. Evoluzione prevista del flusso

Una possibile evoluzione coerente con il progetto è:

1) Order Service crea l'ordine → 2) Order Service chiede il pagamento → 3) Payment Service crea/avvia la transazione → 4) provider elabora → 5) Payment Service aggiorna lo stato → 6) Order Service reagisce all'esito.

In una versione più realistica, il passaggio 4-5 può essere asincrono: il provider invia un webhook e il Payment Service aggiorna la transazione. Successivamente potremo introdurre anche la comunicazione asincrona tra microservizi, quando sarà il momento previsto dal progetto.

11. Punto di partenza per il futuro

La decisione architetturale più importante presa ora è mantenere PaymentProcessor come contratto. La simulazione serve allo sviluppo e ai test; non viene considerata l'implementazione definitiva di un pagamento reale. Quando arriverà l'integrazione con un provider, si sostituirà o si selezionerà l'implementazione del processor tramite configurazione, mantenendo il resto dell'architettura il più stabile possibile.